AI651 Deep Learning for Space, Time and Graphs

Assignment 2

[25280026]

Code repository: `https://github.com/fasih2611/DL4ST-assignment-2`

Fall 2026

1 Question 1: River Forecasting Proposal

I propose a small learned forecaster. I call it RMSTA (reachability-masked space–time attention). Every 10 minutes, it predicts the water level at S1–S4 one hour ahead. Each forecast has three values: the 10th, 50th and 90th percentiles.

The design uses three physical facts: water flows downstream, travel time has a limit, and the along-river distances are known. The model learns the remaining relations from data. These include the actual travel times, the attenuation of flood waves and the rainfall response. When an operational system or a documented flood supports a design choice, I cite it at that choice.

1.1 Architecture diagram

The node set, in flow order, is $V = \{\text{Res}, \text{S1}, \text{T1}, \text{S2}, \text{S3}, \text{S4}, \text{R1}, \text{R2}\}$, so $|V| = 8$. Res is the reservoir. T1 is the station on the monitored tributary. R1 and R2 are the rain gauges.

The model combines the sources in four steps (Figure Q1.1):

1. Block B0 puts every source on one 10-minute grid. The grid ends at the issuance time t_0.
2. Block B1 gives each source type its own linear map to a common width $d = 64$. Levels, rainfall and releases then share one representation.
3. Block B2 groups the sequence of each node into 30-minute tokens. It then puts the tokens of all nodes into one sequence of $L = 386$ tokens.
4. The attention block B3 combines these tokens across nodes and times.

1.2 Assumptions

Supplied facts. The question gives these facts. They are not assumptions.

- F1: The levels at S1–S4 and T1 are recorded every 10 minutes.
- F2: The rainfall at R1 and R2 has timestamps and gauge locations.
- F3: The reservoir releases are recorded. Future release schedules are announced before each forecast, with an announcement time and an effective time.
- F4: The station locations, the river connections and the along-river distances are known.

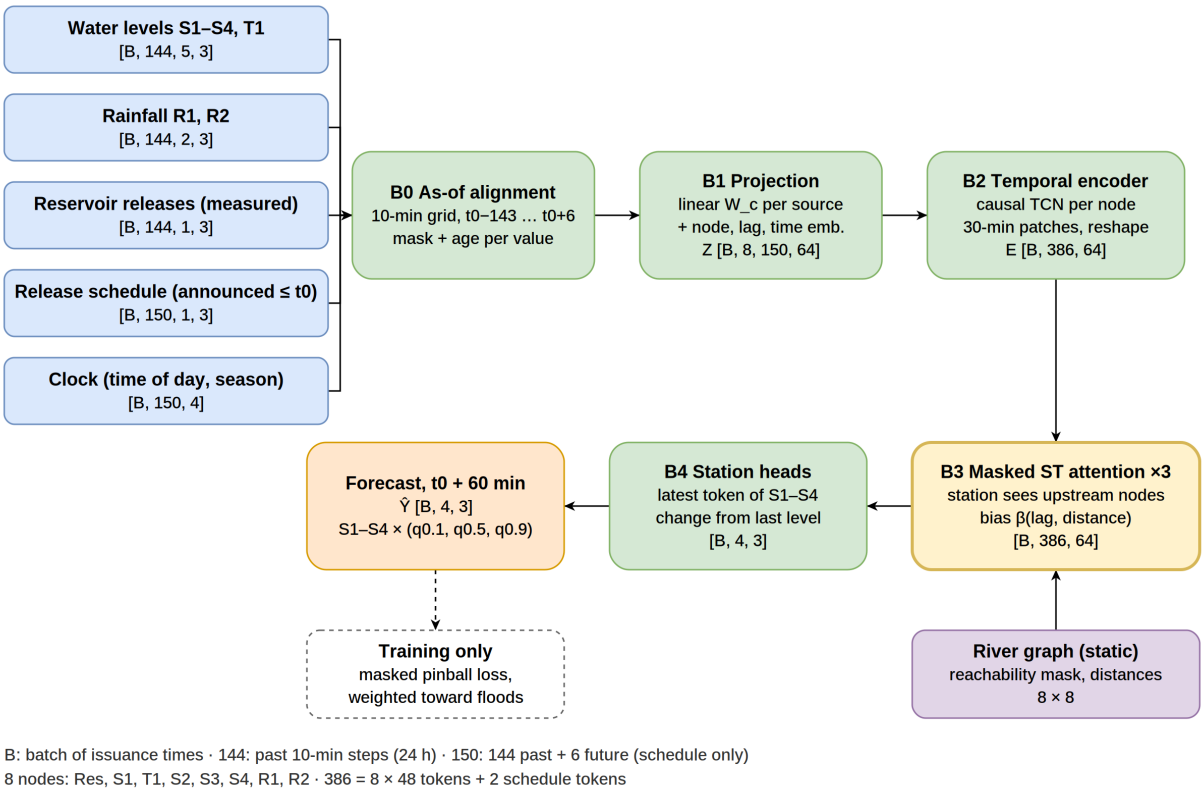


Figure Q1.1: RMSTA architecture. Blue: data sources and their aligned tensor shapes. Green: processing blocks B0–B4. Yellow: the attention block, which combines the sources across space and time. Purple: the static river graph, which supplies the attention mask and the distances. Orange: the forecast. The dashed box applies only to training.

- F5: The travel times and the inflow from the unmonitored tributary are not given.
- F6: Future rainfall and future levels are not available at issuance.
- F7: The stations are not evenly spaced, and travel times change with river conditions.

All items below are my assumptions. For each one, I give the reason and the effect on the design. Assumptions A1–A7 are about the data. Assumptions A8–A11 are about the river.

Assumptions about the data

A1. Record length. At least five years of 10-minute records exist for all stations, with several flood seasons. A water authority with a telemetry network usually has this much history. HydroNets made useful river models from a few years of data (Moshe et al., 2020). Five years would be sufficient to train a model of approximately 0.2 M parameters. I would split the training, validation and test data by year. Then no flood event occurs in both the training data and the test data.

A2. Units and datum. Levels are in metres above a fixed gauge datum. The authority records each datum change. This is standard practice for river gauges. It lets me normalise the levels of each station separately.

A3. Delays and outages. Most telemetry arrives in less than 10 minutes. Delays of up to a few hours and outages affect a few percent of the records. These failures are frequent in remote telemetry. In the 2021 Ahr flood, data transfer stopped and the flood destroyed three gauges (European Flood Awareness System, 2021). Thus every input value has a mask for missing values and an age. Training would simulate delays and outages (Section 1.5).

A4. Rain gauge timing. Each rain gauge reports accumulated rainfall at its own timestamps. Rain gauges and river gauges seldom use the same clock. I add the rainfall into 10-minute bins. I do not interpolate it.

A5. Schedule archive. The authority keeps every revision of the release schedule, with its announcement time and effective time. F3 says that these times exist. With this archive, I can rebuild the information that was known at each past issuance time. I can then train on only that information (Section 1.5).

A6. Sensor noise. The gauge noise is approximately 1 cm. Sensor faults sometimes cause spikes and flat lines. This is typical for pressure sensors and radar level sensors. A quality-control step marks these faulty values as missing.

A7. Warning levels. Each station has an operational warning level. Flood-preparedness agencies usually define such levels. These levels set the loss weights and the evaluation of flood events.

Assumptions about river dynamics

A8. Variable travel time. Travel time changes with flow conditions, as F7 says. Thus the model does not use fixed lags. Its attention weights change with the current state of the river (Section 1.4).

A9. Rain after issuance. Rain that falls after t_0 changes the main-river levels only a little in one hour. The main river usually responds more slowly than that. Small tributaries can respond faster. Thus the model can forecast without future rain. The remaining error makes the quantile band wider. In Section 1.6, I request a rainfall nowcast.

A10. Unmonitored tributary. The flow in the unmonitored tributary follows the rainfall at R2. It also follows T1, but less closely. In the figure, R2 is near that tributary. Neighbouring catchments respond to the same storms. Both rain gauges are visible to all stations. Thus the forecasts at S3 and S4 can use R2 as a proxy for that inflow.

A11. Release record. The release record is the total outflow to the river, and includes the spillway flow. Operators usually record the total release. Releases are a direct input for S1. They reach the downstream stations through the upstream connections.

1.3 Forecasting task and data specification

The time step is $\Delta = 10$ minutes. A forecast can start at any grid time t_0. It uses only the data that arrived by t_0. The history window has $H = 144$ steps, from $t_0 - 143\Delta$ to t_0. This is 24 hours. The horizon is one target time, $t_0 + 6\Delta = t_0 + 60$ minutes.

The only known future input is the reservoir schedule for $t_0 + 1\Delta$ to $t_0 + 6\Delta$. It must be announced at or before t_0. A release that starts after $t_0 + 60$ minutes cannot change the target. Thus the model does not use the schedule after that time. The outputs are the 10th, 50th and 90th percentiles of the level at S1–S4 for $t_0 + 60$ minutes. Table Q1.1 gives all inputs and outputs.

Table Q1.1: Inputs and outputs. B is the batch axis (issuance times in a mini-batch; $B = 1$ in operation).

Tensor	Shape	Axis meanings	Time coverage and timestamp rule
X_{lvl} (input)	$[B, 144, 5, 3]$	batch; time step; station {S1, T1, S2, S3, S4}; channel {normalised level, mask m, age a}	$t_0 - 143\Delta$ to t_0. Each slot holds the reading timestamped in $(\tau - \Delta, \tau]$ that arrived by t_0. If there is none, the last valid value is kept with $m = 0$ and a set to its age.

Tensor	Shape	Axis meanings	Time coverage and timestamp rule
X_{rain} (input)	$[B, 144, 2, 3]$	batch; time step; gauge {R1, R2}; channel {log(1 + accumulation), m, a}	Rain accumulated in $(\tau - \Delta, \tau]$, from reports that arrived by t_0.
X_{rel} (input)	$[B, 144, 1, 3]$	batch; time step; reservoir; channel {measured release, m, a}	Past only, up to t_0.
X_{sch} (input)	$[B, 150, 1, 3]$	batch; time step; reservoir; channel {scheduled release, known flag, time since announcement}	$t_0 - 143\Delta$ to $t_0 + 6\Delta$. Each slot holds the latest revision announced by t_0 that covers that time.
X_{cal} (input)	$[B, 150, 4]$	batch; time step; sine and cosine of time of day and day of year	Same grid as X_{sch}.
A_{up}, D (static)	$[8, 8]$, $[8, 8]$	node × node: upstream reachability (0 or 1), signed along-river distance (km)	Constant; built from F4.
Y (training target)	$[B, 4]$	batch; station {S1, S2, S3, S4}	Level at $t_0 + 6\Delta$.
M_y (target mask)	$[B, 4]$	1 if the target passed quality control	Same time as Y.
$\hat{Y}$ (output)	$[B, 4, 3]$	batch; station; quantile {0.1, 0.5, 0.9}	Valid for $t_0 + 60$ minutes.

Alignment across sampling times. Each source goes on the same grid with its own rule:

- A level slot contains the last observation in its 10-minute bin.
- A rainfall slot contains the sum of the accumulations in its bin.
- A release slot contains the average release rate over its bin.

A value never goes into a slot before its own timestamp. Thus no future information gets into the past part of the window.

Alignment across locations. I do not use the station index as a distance, because the stations are not evenly spaced (F7). Spatial relations enter the model only through the reachability mask and the along-river distances D.

Information at issuance. At t_0, the model has:

- all levels, rainfall and measured releases that arrived by t_0;
- all schedule revisions announced by t_0;
- the static river graph.

It does not have future rainfall or levels, future measured releases, schedule revisions announced after t_0, or observations that arrive after t_0.

1.4 Model computations and design justification

Notation. $v \in V$ is a node and τ is a grid time. $x_{v,\tau}$ is the aligned feature vector from Section 1.3. $c(v)$ is the source type of node v: level, rain or release. $\text{Up}(v)$ is the set of nodes upstream of v. $\text{Dn}_1(v)$ is the next station downstream. A token $n = (v, p)$ is node v in the 30-minute patch p, and t_p is its time.

Core equations

B1, projection. This step maps each aligned input to the common width d:

$$z_{v,\tau} = W_{c(v)}\, x_{v,\tau} + b_{c(v)} + e_v + \pi(\tau - t_0) + \kappa(\tau). \tag{1}$$

e_v is a node embedding. It encodes the node type and the position along the river. π is a learned embedding of the lag from t_0. The lag is negative for the history and positive for the schedule. κ encodes the time of day and the day of the year.

B2, temporal encoding and patching. A causal temporal convolution (TCN) encodes the sequence of each node. It has dilations 1, 2, 4 and 8, and all nodes share it. Then each group of three 10-minute steps becomes one 30-minute token:

$$u_{v,\cdot} = \mathrm{TCN}(z_{v,\cdot}), \qquad e^{(0)}_{v,p} = W_P\,[u_{v,3p-2};\, u_{v,3p-1};\, u_{v,3p}]. \tag{2}$$

This gives 8×48 past tokens and 2 future schedule tokens, so $L = 386$ tokens of width d. The projection comes before the TCN because the sources have different channels and units. When all sources are in one shared space, one TCN can process every node. The TCN comes before the patching. Then it calculates short-term changes, such as the rate of rise, at the full 10-minute resolution.

B3, masked attention. Three pre-normalisation transformer layers combine the tokens. Each layer has 4 heads h:

$$\alpha^h_{nn'} = \mathrm{softmax}_{n'}\left(\frac{(W^h_Q e_n)^\top W^h_K e_{n'}}{\sqrt{d/4}} + \beta^h(t_p - t_{p'},\, D_{v'v}) + \log M_{vv'}\right), \tag{3}$$

$$M_{vv'} = \mathbf{1}\big[v' \in \mathrm{Up}(v) \cup \{v\} \cup \mathrm{Dn}_1(v) \cup \{\mathrm{R1}, \mathrm{R2}\}\big]\,. \tag{4}$$

β^h is a small learned function of the time lag and the river distance between two nodes. Thus it is a learned prior for travel time. The query changes with the current state. Thus the lags that the model uses can change with the flow conditions (A8). M is the node mask, and $\log M$ is $-\infty$ for blocked pairs. The mask uses upstream reachability, not only direct neighbours. This follows the proposal of Wang et al. (2024) for river networks.

B4, forecast mapping. For each station $i \in \{1, \ldots, 4\}$, g_i is the final token of S_i at the latest patch. $\tilde{h}_i$ is its last valid level. σ_i is its standard deviation in the training data. A small network f_i for each station gives the predicted change:

$$\begin{aligned} \hat{y}_{i,0.5} &= \tilde{h}_i + \sigma_i f_i(g_i)_1, \\ \hat{y}_{i,0.9} &= \hat{y}_{i,0.5} + \sigma_i\, \mathrm{softplus}(f_i(g_i)_2)\,, \\ \hat{y}_{i,0.1} &= \hat{y}_{i,0.5} - \sigma_i\, \mathrm{softplus}(f_i(g_i)_3)\,. \end{aligned} \tag{5}$$

Thus the model predicts the change from the last level, not the level itself. If the network output is zero, the forecast is the last level. This persistence forecast is a strong baseline at a one-hour horizon. The softplus terms keep the three quantiles in the correct order.

Training objective. The training would use a masked pinball loss with a flood weight. I give it in two steps. The first step gives a score to one forecast b at one station i. It adds the pinball errors of the three quantiles:

$$\ell_{b,i} = \sum_{q \in \{0.1,\, 0.5,\, 0.9\}} \rho_q(y_{b,i} - \hat{y}_{b,i,q})\,, \qquad \rho_q(u) = \max(qu,\, (q-1)u)\,. \tag{6}$$

The pinball error ρ_q multiplies an under-prediction by q and an over-prediction by $1 - q$. For $q = 0.9$, a forecast that is too low costs nine times more than a forecast that is too high. This moves that output up to the 90th percentile. For $q = 0.5$, both directions cost the same, and the output is the

median. The second step calculates the average of these scores over all valid targets. It gives flood periods a larger weight:

$$\mathcal{L} = \frac{\sum_{b,i} m_{b,i}\, w_{b,i}\, \ell_{b,i}}{\sum_{b,i} m_{b,i}}. \tag{7}$$

This loss uses existing components. I did not invent it. ρ_q is the check loss of quantile regression (Koenker and Bassett, 1978). Its minimum is at the q-th conditional quantile. The sum over several quantiles for one neural forecaster follows MQ-RNN (Wen et al., 2017) and the Temporal Fusion Transformer (Lim et al., 2021). The mask for missing targets follows the masked error for the METR-LA traffic-sensor benchmark (Li et al., 2018). Those sensors also have missing readings. Here, the mask $m_{b,i}$ is M_y. My only change is the flood weight $w_{b,i} = 1 + \lambda\, \mathbf{1}[\text{flood}_{b,i}]$. $\text{flood}_{b,i}$ is true when the level is above the warning level or rises quickly. λ is approximately 4 (A7). I would select the model with the validation loss on held-out years.

Design justification

Direct connections instead of stacked convolutions. A forecast at S4 can need a release from many hours before. A release 12 hours before is 72 steps back and five graph hops upstream. A space–time convolution with kernel size 3 adds two time steps per layer. Thus it would need approximately 36 layers to reach that far. It would also treat neighbouring indices as close, but the stations are not evenly spaced. Attention connects each station to every upstream node in one layer. The relay experiment in the companion notebook showed this difference. The convolution stack lost information outside its reach, but attention did not.

Published work gives the same result. Kirschstein and Sun (2024) found that GNNs with river-network adjacency gave no measurable improvement on the LamaH-CE dataset. Wang et al. (2024) said that the cause is the many hops in a tree-shaped graph. In their results, direct links from each node to all upstream nodes gave 24-hour forecasts. These forecasts were approximately as accurate as the 14-hour forecasts of an EA-LSTM. The cost of attention is small here. With $L = 386$ and $d = 64$, attention mixing costs $2L^2 d \approx 19\,\text{MFLOP}$ per layer. This is sufficiently small to run every 10 minutes (Section 1.5).

Why a mask. The mask adds physical structure to a model with limited training data (A1). Water at S3 cannot change S1. But the results of Kirschstein and Sun (2024) show that topology does not always help. Thus I would compare the masked model with an unmasked model during validation.

Why quantiles and not one value. The unmonitored tributary and missing data cause uncertainty. The same inputs can come before different levels. Thus one forecast value cannot show all possible outcomes. The notebook showed that MSE then predicts the mean and MAE predicts the median. Neither value shows how high the level can go. The pinball loss gives each quantile directly. Its 0.5 quantile is the MAE solution. A flood warning needs the 90th percentile.

Why flood weighting. Most of the record has low flow. Without a weight, these periods would control the loss. But they are not important for warnings.

How each source reaches each forecast

Upstream stations reach a forecast through attention to their tokens. β gives the learned lags. S2 uses S1. S3 uses S1 and S2. S4 uses S1–S3. The tributary station T1 is upstream of S2. Thus S2 and all stations below it can use T1.

The model cannot observe the unmonitored tributary. It uses the rainfall at R2 (A10) as a proxy. It also uses the part of the history of S3 that S2 does not explain. Thus I expect wider bands at S3

and S4. All stations can use the rainfall at R1 and R2, with a learned response lag. This rainfall is most important for S2–S4.

Past releases enter through the reservoir tokens. These tokens are upstream of every station. They reach S1 directly and the other stations after a lag. Announced releases enter through the two future reservoir tokens. They mainly affect S1. They affect S2 only if the travel time from the reservoir is less than one hour.

Physical assumptions and learned mappings

The design sets five physical properties:

- The mask M gives the flow direction and the reachability.
- The 24-hour window gives the limit on travel time.
- β uses the along-river distance as an input (F4).
- The time grid in B0 gives only the data that is available at issuance.
- Persistence is the starting point, because the model predicts the change in level.

The model would learn all other relations from data:

- the travel time as a function of state and distance, through β and the attention scores;
- the attenuation of flood waves between stations;
- the rainfall–runoff response and the contribution of the unmonitored tributary;
- the relation between release and level;
- the width of the uncertainty band.

1.5 Operation and exceptional conditions

(i) Missing observations. Quality control marks gaps and faulty readings as missing ($m = 0$). The model keeps the last valid value and its age. During training, I would remove data at random: whole sensors, blocks of one to six hours, and the recent history of the target station. Then the model learns to use the upstream and neighbouring stations instead. This is important because sensors fail during floods (A3). The loss would ignore missing training targets through M_y. I would not fill them in.

In operation, the forecast continues when the own feed of a station is more than three hours old. But it has a wider band and a quality flag. If more than half of the inputs are missing, the system uses a linear regression on lagged upstream levels. If that also fails, it uses persistence. The operational Google system also used linear models together with LSTMs for stage forecasts (Nevo et al., 2022).

(ii) Delayed observations. The forecast does not wait for late data. It uses only the data that arrived by t_0. A value that arrives after t_0 is not lost. It goes into its correct slot at the next issuance. Thus the history becomes correct again after one cycle. The age channel tells the model the age of each latest value. Training would use the recorded arrival times again. If these records are not available, it would use assumed delays (Section 1.6). Then the model sees the same data ages in training and in operation.

(iii) Announced future releases. The latest revision announced by t_0 gives the schedule for $t_0 + 1\Delta$ to $t_0 + 6\Delta$. The model ignores revisions announced after t_0 until the next issuance. Training

must use the same rule. I would rebuild the inputs from the revision archive (A5). Then the model never sees an announcement before its announcement time. Without this rule, the test results would be better than the results in service.

The two timestamps are important because announcements do not always come through regular channels. For example, in August 2025, India warned Pakistan about high flows on the Tawi and the Sutlej. The warning came through diplomatic channels, not through the Indus Waters Commission (Asia News Network, 2025).

(iv) Differences between scheduled and actual releases. For past slots, the model uses the measured release. The schedule stays as a separate channel. Thus the model can see the recent difference between the two values. It can learn how much to trust the schedule. Training should include the real cases where the two values were different. It should also include random changes of 10–30% in rate and 10–20 minutes in time.

Gate telemetry can show a change that nobody announced. The model then uses the measured value at the next issuance. This can occur. After the 2018 Kerala floods, the main criticism was that the Idukki dam opened its shutters without a warning (The News Minute, n.d.). Each issuance also runs the model with the schedule multiplied by 0.8 and by 1.2. The difference shows operators how much S1 and S2 change with the release.

Feasibility of running every ten minutes. The model has approximately 0.2 M parameters, with 3 layers of width $d = 64$. My estimate is less than approximately 0.25 GFLOP per forecast. With the two scenario runs, it is less than 1 GFLOP. This takes some milliseconds on one CPU core. Quality control and alignment process only eight short series. Their cost is smaller. Retraining would occur offline, once per season or when monitoring finds drift. Thus the arrival of data controls the 10-minute cycle, not the computation.

1.6 Uncertainty, limitations, and additional information

Uncertainties and limitations

Unmonitored tributary. The tributary between S2 and S3 adds inflow that S3 and S4 cannot see. This occurs mostly during local storms over that catchment. Thus the forecasts can be too low when the levels rise. R2 and the history of S3 are proxies, and the bands at S3 and S4 would be wider. I would give the interval coverage for each station. A storm over that catchment that misses R2 is still a problem.

Changing travel times. A model with fixed lags would give peaks at the wrong time. Here, attention changes with the current state. I would measure the error in peak time on held-out events. Travel speeds outside the observed range are still a problem.

Floods larger than any in the record. The learned mapping can predict too low a level for such floods. In the 2021 Ahr flood, the forecast 24 hours ahead gave 5.74 m at Altenahr. The reconstructed peak was approximately 10.2 m (Natural Hazards and Earth System Sciences, 2025). My response is the flood weight, a flag when inputs are outside the historical range, and a wider band. Without hydraulic data, this response is only partly sufficient.

No future rainfall. Rain after t_0 can raise small tributaries in one hour. The quantile band includes this error (A9). A nowcast would make it smaller.

Only two rain gauges. A storm cell can miss both gauges. This is a known problem. In 2018, the Periyar basin in Kerala had 6 of the 32 necessary gauges (Comptroller and Auditor General of India, 2021). Training would treat this as noise. Radar or satellite rainfall would help.

Late or changed release information. After a gate change that nobody announced, the forecasts at S1 and S2 can be incorrect for one hour. The measured release replaces the schedule,

and the scenario runs show the sensitivity. A change that is not announced and not yet measured is still a problem.

Sensor faults, delays and outages. These make the inputs old. In the worst case, the own feed of a station stops. The mask and age channels, the training with removed data, the fallbacks and the quality flags help with this problem.

Channel changes and datum shifts. These cause a slow drift in the forecasts. Drift monitoring and retraining correct it.

Short record with few floods. With few floods in the record, the estimates of the upper quantiles are poor. I would check the coverage for each flow regime. Pretraining on other basins is possible. An LSTM trained on approximately 5,680 gauges has forecast extreme floods in basins without gauges (Nearing et al., 2024).

Additional information requests

Schedule revision archive. An archive of schedule revisions with announcement times would let me train on only the information available at issuance. I could also learn how reliable the schedules are. Without the archive, the system still operates. But I would use the actual release plus synthetic error as the schedule. Then the offline scores would be too good.

Telemetry arrival logs. Arrival logs and quality-control flags would give a realistic simulation of delays and better quality control. Without them, I would use an assumed delay distribution.

Rainfall nowcasts or radar. These would decrease the error from rain after t_0, mostly for the tributaries. Without them, the system still operates, because it does not need future rain.

Rating curves, cross-sections and spillway tables. These would allow discharge inputs and physical routing. They would also give better results in extreme floods. Without them, the model operates with levels.

A gauge on the unmonitored tributary. A gauge, or at least the catchment area of the tributary, would decrease the largest uncertainty at S3 and S4. Without it, the system still operates, with wider bands at those stations.

Warning levels and records of past floods. These would set the loss weights and allow an evaluation of flood events. Without them, I would use percentile thresholds.

Reservoir storage level and spillway status. These would help to predict uncontrolled spill when the reservoir is full. Without them, the system still operates, but the forecasts at S1 are less accurate during extreme events.

Log of river works and gauge changes. This would separate real changes in the river from measurement errors. Without it, the system still operates.

2 Question 2: Motion or Memory?

2.1 Setup

Table 1: Checks of the transformation and window functions that all experiments use (synthetic tensors).

	Check	Expected	Yours
(a)	$\max \lvert M(M(x)) - x \rvert$	0	0, `torch.equal` = True
(b)	$\max \lvert R^4(x) - x \rvert$	0	0, `torch.equal` = True
(c)	$\max \lvert MRM(x) - R^{-1}(x) \rvert$	0	0, `torch.equal` = True
(d)	bright pixel of $M(p)$ (frame, row, column)	(7, 10, 43)	(7, 10, 43)
(e)	bright pixel of $R(p)$ (frame, row, column)	(7, 43, 10)	(7, 43, 10)
(f)	1× input / target frames	1–5 / 6–10	1, 2, 3, 4, 5 / 6, 7, 8, 9, 10
(g)	2× input / target frames	1, 3, 5, 7, 9 / 11, 13, 15, 17, 19	1, 3, 5, 7, 9 / 11, 13, 15, 17, 19
(h)	3× input / target frames	1, 4, 7, 10, 13 / 16, 19	1, 4, 7, 10, 13 / 16, 19

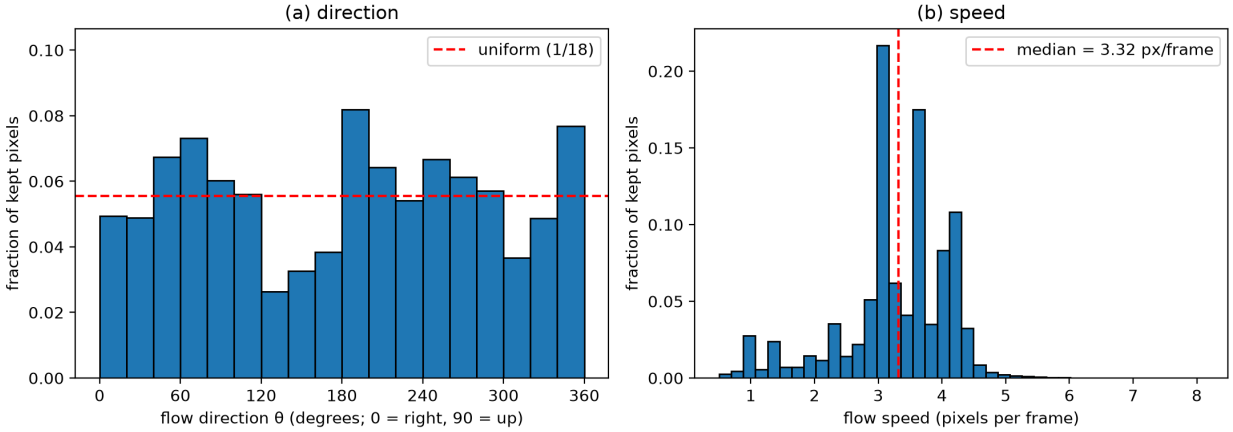


Figure 1: Dense Farneback optical flow on the first 50 training videos (950 consecutive frame pairs). The histograms use the 103,050 kept pixels: intensity above 0.2 in both frames and flow magnitude above 0.5 pixels. (a) Flow direction $\theta = \text{atan2}(-d_y, d_x)$ as a fraction of the kept pixels, in 20° bins. The dashed line is the uniform level 1/18. (b) Flow speed as a fraction of the kept pixels. The median speed is 3.32 pixels per frame.

Q2.1 In Figure 1(a), each 20° direction bin contains between 0.026 and 0.082 of the kept pixels. The uniform level is 0.056. The histogram is not smooth, but all directions are present. A mirror changes θ to $180° - \theta$. A 90° rotation changes θ to $\theta + 90°$. Both transformations only move the bins of the histogram, and every bin already has many pixels. Thus mirrored and rotated motion is familiar to a model that trains on this data. The histogram is not smooth because the sample is small. The 50 videos contain only 100 digit paths, and each digit moves in a fixed direction between bounces.

A typical speed is 3.32 pixels per frame. 90% of the moving pixels are slower than 4.12 pixels per frame. At 2× and 3×, the same digits move approximately 6.6 and 10 pixels per frame. In the training data, only 0.46% of the moving pixels are faster than 5 pixels per frame. Only 0.02% are faster than 6.6. Thus faster motion is new to the model.

Overlap does not change these answers. I marked a frame pair as overlapping when a thresholded frame did not contain two separate digits. This applied to 461 of the 950 pairs. Without these pairs, the median speed changed from 3.32 to 3.26 pixels per frame. No direction bin changed by more than 0.0074.

Q2.2 The mirror and the rotation change the appearance of the digits. A mirrored 2 is not a 2, and a rotated digit is even less familiar. But Q2.1 shows that they do not change the motion statistics. They only move directions between bins that already have many pixels, and they do not change the speeds. The speed-up has the opposite effect. The digits look the same, but they move two or three times farther between frames. This is new motion.

Thus the three transformations make a controlled experiment. A failure under the mirror or the rotation shows a problem with appearance. A failure under the speed-up shows a problem with motion. All three transformations use the same test videos. Thus the comparison keeps all other conditions the same.

2.2 Baseline: ConvLSTM

Table 2: Model A and the two reference predictors (test split, 1×, $t = 1$). Entries for model A show mean (seed 0, seed 1).

	Model A	**Blank**	**Copy-last**
MSE_1	0.0169 (0.0173, 0.0165)	0.0426	0.0492
MSE_2	0.0202 (0.0207, 0.0196)	0.0426	0.0604
MSE_3	0.0224 (0.0230, 0.0218)	0.0426	0.0669
MSE_4	0.0241 (0.0247, 0.0235)	0.0425	0.0708
MSE_5	0.0257 (0.0263, 0.0251)	0.0425	0.0732
$E^{(5)}$	0.0219 (0.0224, 0.0213)	0.0426	0.0641
Trainable parameters	480,784	0	0

Q2.3 The reach of model A is 8 original pixels per time step. After `pixel_unshuffle`$(\cdot, 4)$, one cell of the 16×16 grid is a 4×4 patch. Thus one cell is 4 pixels, from centre to centre. Only the two 3×3 gate convolutions move information. Each convolution moves it one cell or less.

The longest path in one step starts at the new frame u_τ:

1. The convolution of layer 1 over $[u_\tau, h^{(1)}_{\tau-1}]$ moves it one cell into $h^{(1)}_\tau$.
2. The convolution of layer 2 over $[h^{(1)}_\tau, h^{(2)}_{\tau-1}]$ moves it one more cell into $h^{(2)}_\tau$.
3. The 1×1 readout and the pixel shuffle do not move it.

This gives 2 cells, or 8 pixels. A path from $h^{(1)}_{\tau-1}$ also goes through both convolutions (8 pixels). $c^{(1)}_{\tau-1}$ enters element by element and then goes through layer 2 only (4 pixels). $h^{(2)}_{\tau-1}$ goes through layer 2 only (4 pixels). $c^{(2)}_{\tau-1}$ does not move. The recurrence can move information farther over several steps. But in one update, the limit is 8 pixels.

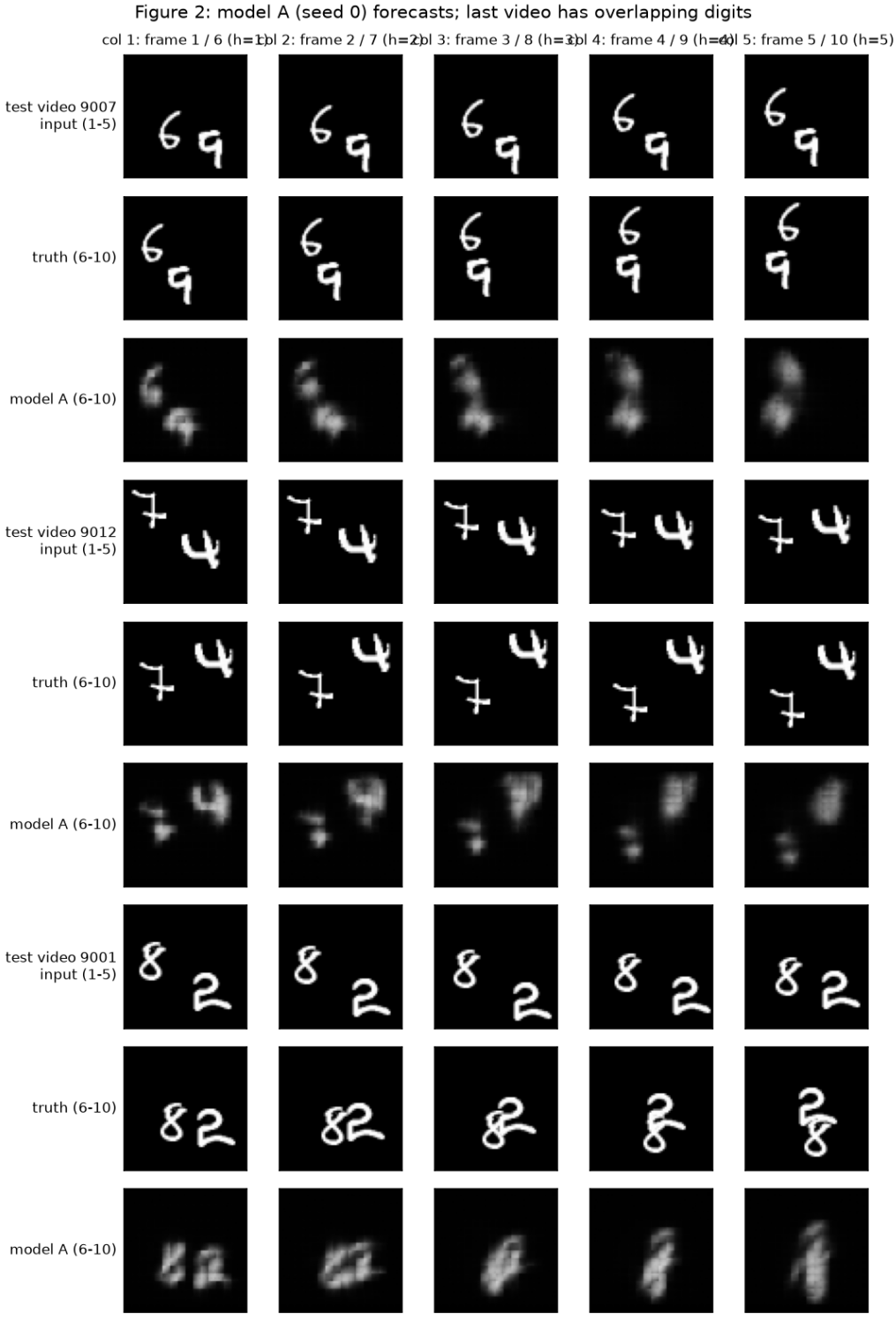


Figure 2: Model A (seed 0) on three test videos. For each video, the rows show the input frames 1–5, the true frames 6–10 and the predicted frames 6–10. The digits in videos 9007 and 9012 stay apart. In video 9001, the 8 and the 2 overlap in the true future.

Q2.4 Model A has approximately half the error of the reference predictors. Its $E^{(5)}$ is 0.0219. Blank has 0.0426 and Copy-last has 0.0641 (Table 2). Copy-last is worse than an empty frame. When a digit moves a few pixels, a copied frame has two errors. It has ink where the digit was, and it has no ink where the digit is now.

The error of model A increases with the forecast horizon. It is 0.0169 at $h = 1$ and 0.0257 at $h = 5$, approximately 50% higher. Figure 2 shows the cause. Each prediction is the input for the next step. The digits stay in the correct positions, but they become more blurred at each step. When digits overlap (video 9001), the 8 and the 2 become one blurred blob. After the digits separate, neither digit is recognisable.

Q2.5 I predict that model A will pass the mirror test and the rotation test. Convolutional networks look for symmetry. I predict that the speed test will be too difficult for model A.

2.3 Testing the baseline

Mirror and rotation

Table 3: Mirror and rotation tests (test split, 1×, $t = 1$). Entries show mean (seed 0, seed 1).

	Model A	**Blank**	**Copy-last**
$E^{(5)}$	0.0219 (0.0224, 0.0213)	0.0426	0.0641
C_M	0.00181 (0.00157, 0.00205)	0	0
E_M	0.0225 (0.0230, 0.0220)	0.0426	0.0641
C_R	0.00289 (0.00286, 0.00292)	0	0
E_R	0.0235 (0.0240, 0.0231)	0.0426	0.0641

Q2.6 Blank and Copy-last have $C_M = C_R = 0$ exactly. An empty frame or a copied frame does not change when you rearrange its pixels. But these two predictors have the largest errors in Table 3. Thus a small C_T shows only that the forecaster treats a scene and its transformed copy in the same way. It does not show that either prediction is correct. Thus E_T must always be given with C_T. A model can be fully consistent and fully wrong.

Q2.7 Video 9380 under rotation is the third panel of Figure 3. In rows 3 and 4, the two digits move along the same paths. Thus the model predicts the same motion. The change is in the digits. In row 4, the rotated original prediction keeps the 8 on its side. In row 3, the model sees the rotated input directly. By $h = 4$–5, it draws the 8 as an upright vertical blob, and the rotated 3 loses its shape. The model changes unfamiliar shapes towards the upright digits from its training data. This causes the large C_R of this video (0.00790, the largest in the test set).

Q2.8 Rotation is more difficult than mirroring for model A. C_R is 0.00289 and C_M is 0.00181. E_R is 0.0235 and E_M is 0.0225. Both are larger than the error on original clips, $E^{(5)} = 0.0219$ (Table 3). The motion does not cause the difference. Q2.1 shows that both transformations only move directions between bins with many pixels. They do not change the speeds. Thus mirrored motion and rotated motion are equally familiar. The cause is appearance. A rotated digit looks less

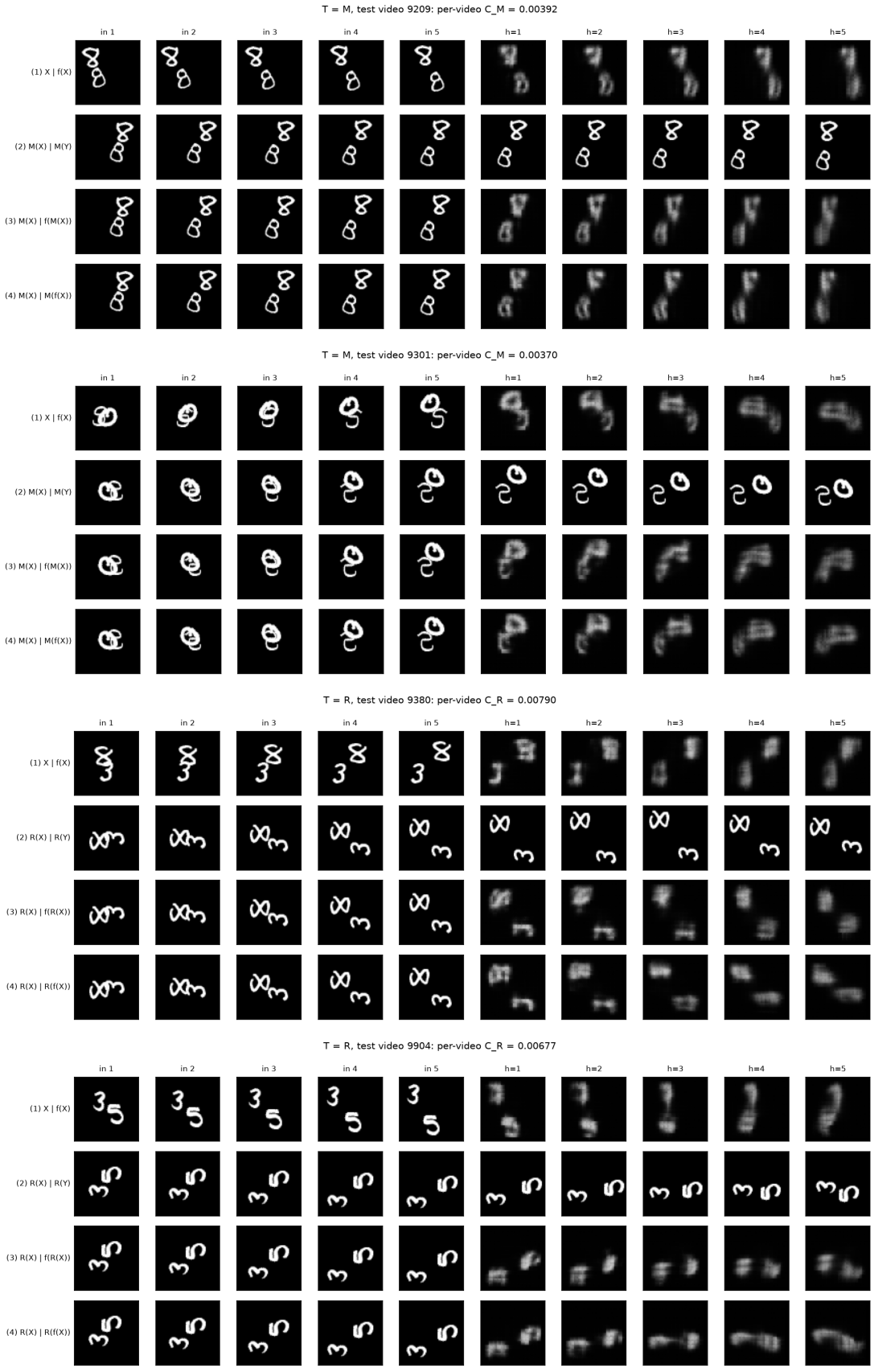


Figure 3: The two test videos with the largest per-video C_M (9209, 9301; $T = M$) and the two with the largest per-video C_R (9380, 9904; $T = R$), for model A, seed 0. Rows: (1) $X \mid f(X)$; (2) $T(X) \mid T(Y)$; (3) $T(X) \mid f(T(X))$; (4) $T(X) \mid T(f(X))$.

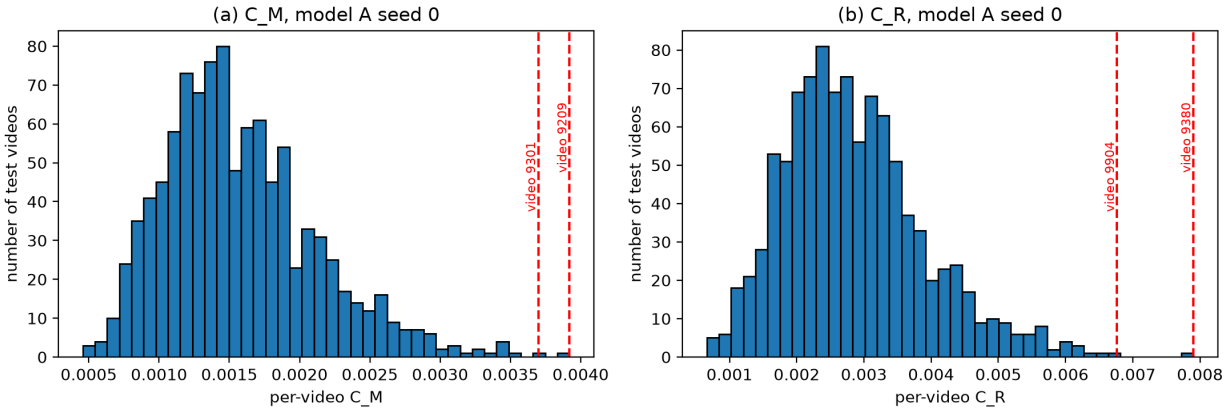


Figure 4: Per-video consistency gaps of model A (seed 0) over the 1,000 test videos: (a) C_M, (b) C_R. The dashed lines show the videos in Figure 3.

like a training digit than a mirrored digit. Q2.7 shows that the model changes the appearance of rotated digits. Model B confirms this result (Q2.13).

Faster motion

Table 4: Faster motion (test split, $t = 1$). Entries show mean (seed 0, seed 1).

	Model A	**Copy-last**
$E^{(2)}$ at 1×	0.0186 (0.0190, 0.0181)	0.0548
$E^{(2)}$ at 2×	0.0444 (0.0437, 0.0451)	0.0661
$E^{(2)}$ at 3×	0.0509 (0.0505, 0.0512)	0.0703
$E^{(5)}$ at 1×	0.0219 (0.0224, 0.0213)	0.0641
$E^{(5)}$ at 2×	0.0490 (0.0477, 0.0503)	0.0719

Q2.9 The predictions of model A fail at 2×. Its $E^{(2)}$ increases from 0.0186 at 1× to 0.0444 at 2×. This is worse than an empty frame (Blank has $E^{(2)} = 0.0426$). At 3×, it is 0.0509 (Table 4).

This result does not agree with my reach estimate. Motion at 2× is approximately 6.6 pixels per frame. This is in the 8-pixel reach. Thus the reach predicted a failure only at 3× (approximately 10 pixels). The estimate missed one point: reach is necessary, but it is not sufficient. Reach shows that information can move 8 pixels in one step. It does not show that the trained weights move a digit that far. Model A saw only displacements of approximately 3 pixels in training (Figure 1).

Figure 5 shows the failure. At 2× and 3×, the 1 stops near its last input position. It does not jump forward. Thus the model continues the motion at the speed from its training data. The 4 becomes a horizontal streak. This blur shows a model that does not know the position of the digit. It spreads the ink over the possible positions.

Q2.10 The results of model A alone cannot separate the two causes. Motion at 2× and 3× is new to the model, and motion at 3× is also outside its reach. The experiment must separate the two causes:

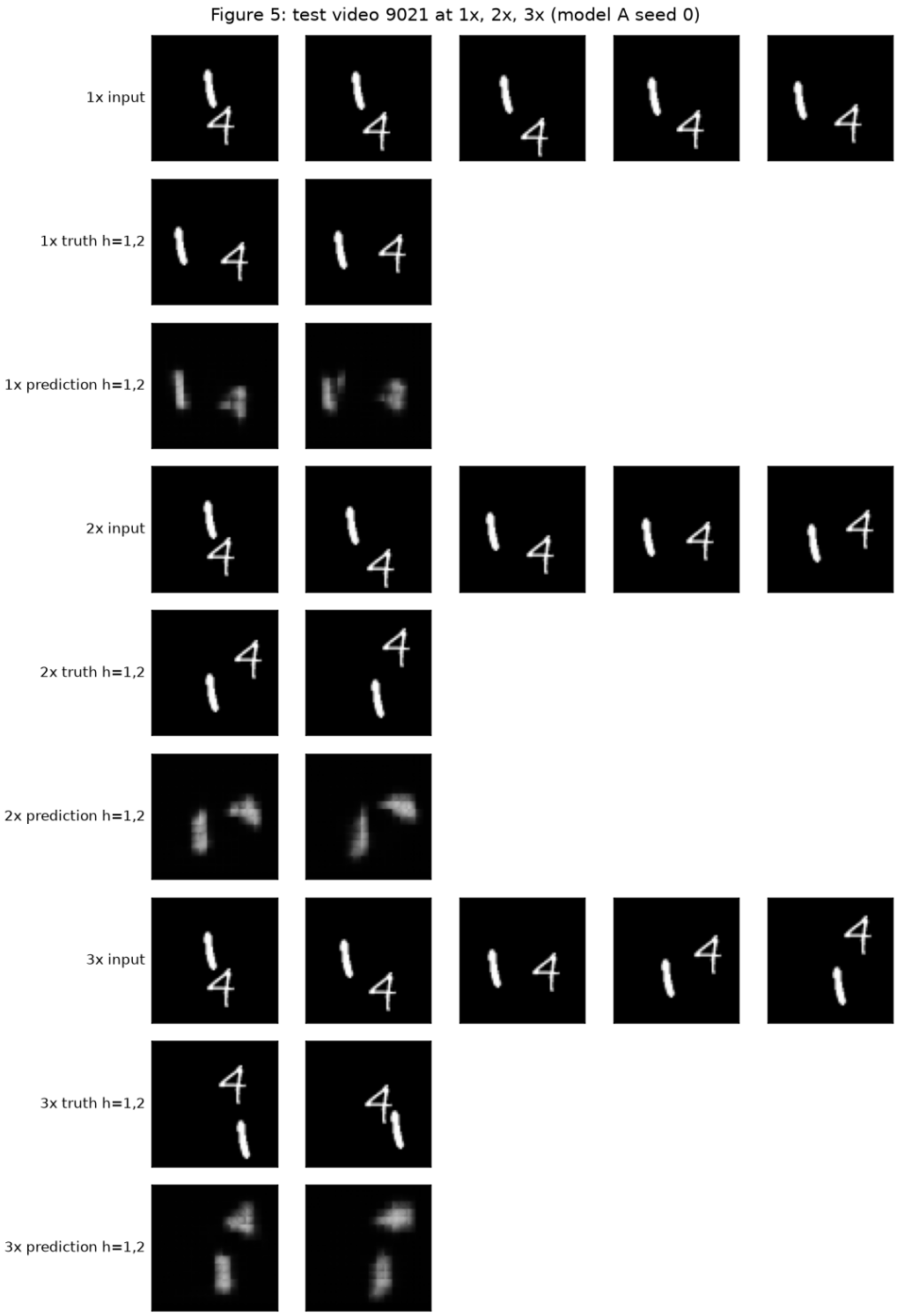


Figure 5: Test video 9021 at 1×, 2× and 3× ($t = 1$). The rows show the five input frames, the first two true future frames and the first two predictions of model A (seed 0). The digits do not overlap in the true frames that the figure shows.

1. Train the same architecture on windows at 1×, 2× and 3×. If the cause was new motion, this model improves wherever the motion is in its reach.

2. Train an architecture with a larger reach on the same data. If this model improves at 3× and the first model does not, the remaining cause was reach.

Models C and D below do these two steps.

2.4 What can data fix?

Q2.11 I predict that mirror and rotation augmentation will improve the results of the mirror test and the rotation test. This augmentation will not improve the results at higher speeds. I predict that speed augmentation will give the best results of the three changes.

Table 5: Models A–D (test split, $t = 1$). Entries show mean (seed 0, seed 1).

	A	**B**	**C**	**D**
$E^{(5)}$ at 1×	0.0219 (0.0224, 0.0213)	0.0223 (0.0223, 0.0223)	0.0269 (0.0278, 0.0260)	0.0286 (0.0287, 0.0285)
C_M	0.00181 (0.00157, 0.00205)	0.00143 (0.00144, 0.00142)	0.00146 (0.00171, 0.00121)	9.40×10^{-4} (9.19×10^{-4}, 9.61×10^{-4})
E_M	0.0225 (0.0230, 0.0220)	0.0222 (0.0223, 0.0222)	0.0271 (0.0280, 0.0263)	0.0289 (0.0291, 0.0287)
C_R	0.00289 (0.00286, 0.00292)	0.00162 (0.00165, 0.00160)	0.00255 (0.00269, 0.00242)	0.00181 (0.00174, 0.00187)
E_R	0.0235 (0.0240, 0.0231)	0.0223 (0.0223, 0.0222)	0.0280 (0.0289, 0.0272)	0.0296 (0.0297, 0.0295)
$E^{(2)}$ at 1×	0.0186 (0.0190, 0.0181)	0.0189 (0.0190, 0.0188)	0.0232 (0.0240, 0.0223)	0.0263 (0.0264, 0.0262)
$E^{(2)}$ at 2×	0.0444 (0.0437, 0.0451)	0.0434 (0.0417, 0.0452)	0.0288 (0.0292, 0.0284)	0.0291 (0.0290, 0.0291)
$E^{(2)}$ at 3×	0.0509 (0.0505, 0.0512)	0.0482 (0.0447, 0.0518)	0.0334 (0.0337, 0.0331)	0.0314 (0.0313, 0.0314)
Trainable parameters	480,784	480,784	480,784	480,784
Training time per epoch (s)	2.63 (2.61, 2.64)	2.85 (2.81, 2.89)	2.64 (2.70, 2.59)	3.03 (3.03, 3.03)
Inference time, 1,000 videos (s)	0.100 (0.100, 0.100)	0.099 (0.100, 0.099)	0.100 (0.099, 0.100)	0.091 (0.091, 0.091)

Q2.12 Model D is model A with a dilation of 2 in every ConvLSTM gate convolution. It keeps the other parts of model A: the 3×3 kernels, 64 hidden channels, 4×4 folding, the 9-step rollout and the training protocol. It trains like model C, with speeds at random from 1×, 2× and 3×. Dilation puts one empty cell between the kernel taps and adds no weights. Thus model D has exactly the same 480,784 parameters as model A. This is much less than the limit of 721,176.

I calculated the reach in the same way as in Q2.3. A 3×3 kernel with dilation 2 reads cells up to 2 cells away. On the longest path (new frame to $h_\tau^{(1)}$ to $h_\tau^{(2)}$), each of the two convolutions moves information 2 cells, not 1. This gives 4 cells, or 16 pixels per step. This is two times the 8 pixels of model A. It is also more than the approximately 10 pixels per frame at 3×.

I selected this design on the validation split only. The other candidate used 8×8 folding. It also had a reach of 16 pixels, with 594,496 parameters. I set the selection rule before training: the lowest validation $E^{(2)}$, averaged over 1×, 2×, 3× and both seeds. The dilation design had 0.0286 and the 8×8 folding had 0.0370.

Q2.13 Each failure from Testing the Baseline gets one label:

- *Inconsistency under mirror and rotation, larger for rotation: appearance.* Model B saw the same motion as model A, but with transformed digits. It decreases the rotation gap by almost half (C_R from 0.00289 to 0.00162). Its error on rotated clips ($E_R = 0.0223$) becomes equal to its error on original clips ($E^{(5)} = 0.0223$). Model B does not improve the speed results ($E^{(2)}$ at 2× stays at 0.0434).

- *Failure at 2×: unfamiliar motion.* Model C has the same architecture and reach as model A. It only sees faster motion during training. It decreases $E^{(2)}$ at 2× from 0.0444 to 0.0288, and at 3× from 0.0509 to 0.0334.

- *Remaining error at 3×: partly architectural reach.* Model D is different from model C only in reach. It improves only where the motion is more than the 8-pixel reach of model A. At 3×, its $E^{(2)}$ is 0.0314 and model C has 0.0334. At 2×, the two models are equal (0.0291 and 0.0288).

In these results, model C does not fail at 3×. Thus reach is a smaller cause than unfamiliar motion, but it is a real cause. More training data cannot remove it. Data changes the weights, not the connections. In one update, a pixel more than 8 pixels away has no path to the output. Its gradient is exactly zero. Thus no data can teach model A to use it. The network can only find a fast digit indirectly, over several steps. This is why model C stays worse than model D at 3×.

Q2.14 Yes. Speed augmentation increased the error on the original 1× clips. $E^{(5)}$ is 0.0269 for model C and 0.0286 for model D. Model A has 0.0219. These are increases of 23% and 31%. The training budget is fixed at 30 epochs, and two thirds of the training windows now have faster motion. The dilated kernels of model D also add a grid pattern to its predictions. Symmetry augmentation had almost no cost (model B: 0.0223).

In return, the error at 2× and 3× decreased by 34–38%. Before, it was as bad as an empty frame. After, the forecasts are useful. A forecaster must learn motion, not one speed. Thus I think that this trade is worth its cost. But it is a trade, not a free improvement.

2.5 The final model

For a forecaster f, the symmetrised forecaster is $\bar{f}(X) = \frac{1}{8}\sum_{g\in G} g^{-1}(f(g(X)))$.

Why $C_M = C_R = 0$ exactly. Let $T \in G$. G is a group, so $g \mapsto gT$ is a bijection of G. Substitute $g' = gT$, so that $g^{-1} = Tg'^{-1}$:

$$\bar{f}(T(X)) = \frac{1}{8}\sum_{g\in G} g^{-1} f(gT(X)) = \frac{1}{8}\sum_{g'\in G} T\, g'^{-1} f(g'(X)) = T(\bar{f}(X)).$$

The last step uses the linearity of T. T only rearranges pixels, so it commutes with the average. Thus $\bar{f}(T(X)) = T(\bar{f}(X))$ for $T = M$ and $T = R$. Both consistency gaps are exactly zero for every f. My implementation gives values of approximately 10^{-17}. This is floating-point rounding (Table 6).

My final model is model D with this symmetrisation. I chose model D because it has the lowest error at 3× in Table 5. I did not train the optional D+B model.

Q2.15 Model B learns symmetry, and the symmetry is only approximate. Augmentation decreases its gaps to $C_M = 0.00143$ and $C_R = 0.00162$, but not to zero. No part of the network makes a rotated input give a rotated output. Augmentation only makes this likely for inputs like the training data. The symmetrised models are exact ($C_M, C_R \approx 10^{-17}$) because of the group argument above. This is true for all weights.

Built-in symmetry also improved the accuracy a little. Symmetrised B has $E^{(5)} = 0.0216$, and B has 0.0223. Its error on mirrored and rotated clips is equal to its error on original clips, by construction.

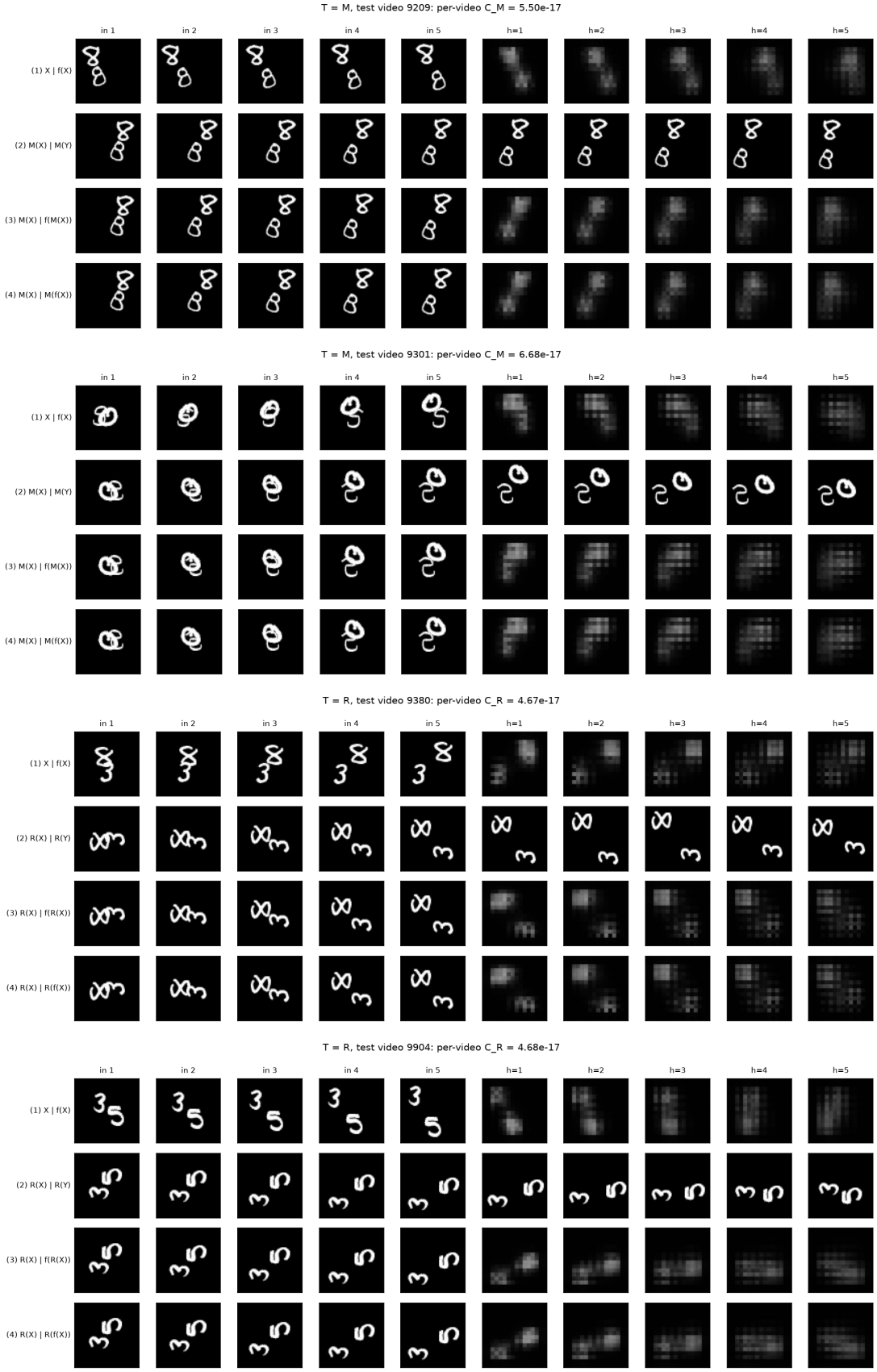


Figure 6: (a) Figure 3 again, with the final model (symmetrised D, seed 0), on the same videos and with the same rows. Part (b) follows.

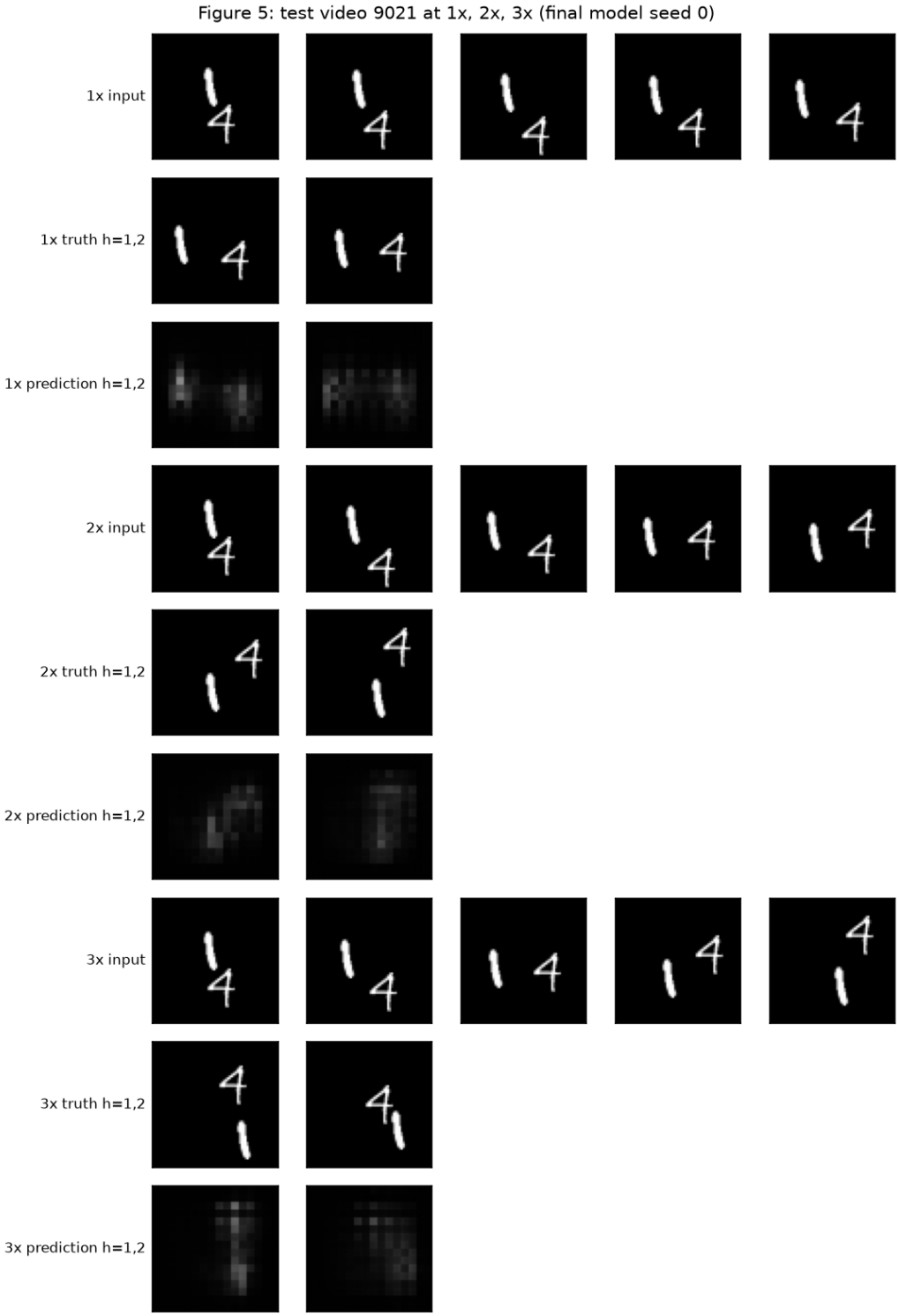


Figure 6: (b) Figure 5 again, with the final model (symmetrised D, seed 0), on the same video, 9021.

Table 6: Symmetrised forecasters (test split, $t = 1$). Entries show mean (seed 0, seed 1).

	Symmetrised B	**Symmetrised D (final)**
$E^{(5)}$ at 1×	0.0216 (0.0216, 0.0215)	0.0285 (0.0286, 0.0284)
C_M	9.03×10^{-17} (8.98×10^{-17}, 9.09×10^{-17})	5.26×10^{-17} (4.55×10^{-17}, 5.98×10^{-17})
E_M	0.0216 (0.0216, 0.0215)	0.0285 (0.0286, 0.0284)
C_R	7.35×10^{-17} (7.33×10^{-17}, 7.37×10^{-17})	4.08×10^{-17} (3.56×10^{-17}, 4.60×10^{-17})
E_R	0.0216 (0.0216, 0.0215)	0.0285 (0.0286, 0.0284)
$E^{(2)}$ at 1×	0.0184 (0.0185, 0.0184)	0.0263 (0.0263, 0.0262)
$E^{(2)}$ at 2×	0.0427 (0.0410, 0.0444)	0.0290 (0.0289, 0.0290)
$E^{(2)}$ at 3×	0.0473 (0.0438, 0.0508)	0.0311 (0.0310, 0.0312)
Trainable parameters	480,784	480,784
Training time per epoch (s)	2.85 (2.81, 2.89)	3.03 (3.03, 3.03)
Inference time, 1,000 videos (s)	0.820 (0.823, 0.817)	0.789 (0.806, 0.773)

The cost is at inference. It needs eight forward passes, not one: approximately 0.8 s, not 0.1 s, for 1,000 videos (Tables 5 and 6). It needs no more training.

An average of eight predictions is blurred when the predictions are different. For example, the base model can draw a rotated digit with a different shape. It can also put a fast digit in different positions in different orientations. Built-in symmetry can decrease accuracy when the base model is strongly specific to one orientation. It can also be a problem when a sharp prediction is more important than the squared error. The squared error prefers averages, because the mean gives its minimum. Here, symmetrisation did not increase the error (symmetrised D 0.0285, D 0.0286). But the frames of the final model are visibly softer than the frames of model A.

Q2.16 Only partly. Video 9380 under rotation is the third panel of Figure 6(a). Rows 3 and 4 are now the same ($C_R = 4.67 \times 10^{-17}$). Thus the shape change from Q2.7, where the model drew the rotated 8 upright, does not occur. Symmetrisation removed it, because the model now predicts all orientations in the same way. Both digits also still move to the correct places.

But the final model does not draw recognisable digits. Its digits are soft blobs with a grid pattern. The dilated convolutions cause the grid pattern, and the average makes the blobs softer. Thus its per-video E_R on this video (0.0411) is higher than that of model A (0.0372). Symmetrisation removed the inconsistency. The dilated architecture and the average caused the loss of sharpness. At 3× (Figure 6(b)), speed augmentation and the larger reach put the ink of the digits near the true positions. Model A left the 1 behind.

Q2.17 (final verdict) My final model learned more motion than model A, but not all motion. It is exactly consistent under mirror and rotation by construction. Speed augmentation and a 16-pixel reach let it follow digits at 2× and 3×. Its 3× error is 0.0311, and model A has 0.0509. Model A stopped the digits at the training speed. The controlled comparisons are the evidence for a sceptic. Model C corrected fast motion with the same architecture. Model D improved only where the motion was more than the old reach. Model B removed the rotation penalty without new motion. The

test that the model still fails is identity. At all speeds, its digits become blurred blobs. Its error on original clips (0.0285) is also worse than that of model A (0.0219). The model knows where the digits go. It does not know what they are when they arrive.

References

Asia News Network. Pakistan's FO confirms India's flood warnings, notes use of diplomatic channel over Indus Waters Treaty. https://asianews.network/?p=222853, 2025.

Comptroller and Auditor General of India. Performance audit on preparedness and response to floods in Kerala. Technical Report No. 6 of 2021, Government of India, 2021.

European Flood Awareness System. The Ahr flood, presentation of 28 october 2021. https://european-flood.emergency.copernicus.eu/sites/default/files/2021-10/20211028_EFAS_AhrFlood_Johst.pdf, 2021.

Nikolas Kirschstein and Yixuan Sun. The merit of river network topology for neural flood forecasting. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *PMLR*, pages 24713–24725, 2024.

Roger Koenker and Gilbert Bassett. Regression quantiles. *Econometrica*, 46(1):33–50, 1978.

Yaguang Li, Rose Yu, Cyrus Shahabi, and Yan Liu. Diffusion convolutional recurrent neural network: Data-driven traffic forecasting. In *International Conference on Learning Representations*, 2018.

Bryan Lim, Sercan Ö. Arık, Nicolas Loeff, and Tomas Pfister. Temporal fusion transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting*, 37(4): 1748–1764, 2021.

Zach Moshe, Asher Metzger, Gal Elidan, Frederik Kratzert, Sella Nevo, and Ran El-Yaniv. HydroNets: Leveraging river structure for hydrologic modeling. *arXiv preprint arXiv:2007.00595*, 2020.

Natural Hazards and Earth System Sciences. Article 25, 581 (2025), reporting the Ahr 2021 forecast and the peak reconstruction of Apel et al. (2022). https://nhess.copernicus.org/articles/25/581/2025/, 2025. Author list to be completed.

Grey Nearing et al. Global prediction of extreme floods in ungauged watersheds. *Nature*, 627: 559–563, 2024.

Sella Nevo et al. Flood forecasting with machine learning models in an operational framework. *Hydrology and Earth System Sciences*, 26:4013–4032, 2022.

The News Minute. Lesson from Kerala floods: Idukki dams get sirens to alert residents on shutter opening. https://www.thenewsminute.com/amp/story/kerala/lesson-kerala-floods-idukki-dams-get-sirens-alert-residents-shutter-opening-111991, n.d.

Hongjun Wang, Jiyuan Chen, Yinqiang Zheng, and Xuan Song. Accelerating flood warnings by 10 hours: The power of river network topology in AI-enhanced flood forecasting. *arXiv preprint arXiv:2410.05536*, 2024.

Ruofeng Wen, Kari Torkkola, Balakrishnan Narayanaswamy, and Dhruv Madeka. A multi-horizon quantile recurrent forecaster. *arXiv preprint arXiv:1711.11053*, 2017. NIPS 2017 Time Series Workshop.

Use of generative AI

I used generative AI to look up and understand the references of Q1, to make the report grounded in past work, AI was also used to format and generate the figures of this report, the suggestion of using β^h as a learned function of the physical river distances and utilizing the pinball loss instead of doing direct predictions.